

Static & Dynamic Code Analysis + GDB Debugging

What's going on here ?

```
#include <iostream>
using namespace std;

int* createArray(int size) {
    int* arr = new int[size];
    for (int i = 0; i <= size; i++) { // Off-by-one error
        arr[i] = i * 2;
    }
    return arr;
}

void processData(int* data) {
    cout << "Processing: " << *data << endl;
    delete[] data;
}

int main() {
    int* numbers = createArray(5);
    processData(numbers);

    int* ptr = nullptr // Wahala : no ";"
    *ptr = 10; // Null pointer dereference

    int* leak = new int[100]; // Memory leak

    cout << "Value: " << numbers[0] << endl; // Use after free

    return 0;
}
```

Let's compile it !!

```
mhd@mhd:~/DSA4E/labs/lab5$ g++ -o code1 code1.cpp
code1.cpp: In function 'int main()':
code1.cpp:22:5: error: invalid operands of types 'std::nullptr_t' and 'int*' to binary 'operator*'
 21 |     int* ptr = nullptr
    |               ~~~~~~
 22 |     *ptr = 10; // Null pointer dereference
    |     ^~~~
mhd@mhd:~/DSA4E/labs/lab5$
```

After fixing the issue by adding the missing “;” the code compiled

```
mhd@mhd:~/DSA4E/labs/lab5$ g++ -o code1 code1.cpp
mhd@mhd:~/DSA4E/labs/lab5$
```

But when we run our executable, WAHALA !!

```
mhd@mhd:~/DSA4E/labs/lab5$ ./code1
Processing: 0
Segmentation fault (core dumped)
mhd@mhd:~/DSA4E/labs/lab5$
```

The Compiler helps but we need more

- ✓ Caught: Syntax error (missing semicolon)
- ✗ Missed: Logical errors, memory issues, null dereference, use-after-free

Static Code Analysis

What is Static Analysis?

- Analyzes source code **WITHOUT** executing it
- Finds potential bugs, coding standard violations, and suspicious patterns
- Catches issues that compilers might miss
- Runs quickly as part of development workflow

Benefits:

- Early bug detection (before runtime)
- Finds code smells and anti-patterns
- Enforces coding standards
- No need to create test cases

Static Analysis Tool - cppcheck

There are many tools out there that can help with static analysis, we will be using cppcheck

What is cppcheck?

- Free, open-source static analysis tool for C/C++
- Detects: null pointer dereferences, memory leaks (simple ones), array bounds errors, uninitialized variables, and more

Installation on Linux:

Ubuntu/Debian

```
sudo apt-get update  
sudo apt-get install cppcheck
```

Fedora/RHEL

```
sudo dnf install cppcheck
```

Verify installation

```
cppcheck --version
```

Basic usage

```
cppcheck mycode.cpp
```

With more parameters

```
cppcheck --enable=all --suppress=missingIncludeSystem mycode.cpp
```

Parameters

`--enable=all` : This option tells cppcheck which categories of checks to run. `--enable=` controls the scope of analysis. So this means Run all available checks, including warnings, style, performance, portability, and information messages.

`--suppress=missingIncludeSystem` : This tells cppcheck to ignore a specific type of warning. `missingIncludeSystem` is a common message that appears when cppcheck can't find system header files (like `<iostream>` or `<vector>`). system headers are often outside your control, so this warning can be suppressed safely

Basic usage

```
cppcheck mycode.cpp
```

With more parameters

```
cppcheck --enable=all --suppress=missingIncludeSystem mycode.cpp
```

Parameters

`--enable=all` : This option tells cppcheck which categories of checks to run. `--enable=` controls the scope of analysis. So this means Run all available checks, including warnings, style, performance, portability, and information messages.

`--suppress=missingIncludeSystem` : This tells cppcheck to ignore a specific type of warning. `missingIncludeSystem` is a common message that appears when cppcheck can't find system header files (like `<iostream>` or `<vector>`). system headers are often outside your control, so this warning can be suppressed safely

Static analysis

```
mhd@mhd:~/DSA4E/labs/lab5$ cppcheck --enable=all --suppress=missingIncludeSystem code1.cpp
Checking code1.cpp ...
code1.cpp:28:5: error: Memory leak: leak [memleak]
    return 0;
    ^
code1.cpp:22:6: error: Null pointer dereference: ptr [nullPointer]
    *ptr = 10; // Null pointer dereference
    ^
code1.cpp:21:16: note: Assignment 'ptr=nullptr', assigned value is 0
    int* ptr = nullptr;
               ^
code1.cpp:22:6: note: Null pointer dereference
    *ptr = 10; // Null pointer dereference
    ^
code1.cpp:24:10: style: Variable 'leak' can be declared as pointer to const [constVariablePointer]
    int* leak = new int[100]; // Memory leak
    ^
code1.cpp:24:15: style: Variable 'leak' is assigned a value that is never used. [unreadVariable]
    int* leak = new int[100]; // Memory leak
    ^
code1.cpp:24:10: style: Variable 'leak' is allocated memory that is never used. [unusedAllocatedMemory]
    int* leak = new int[100]; // Memory leak
    ^
nofile:0:0: information: Active checkers: 161/592 (use --checkers-report=<filename> to see details) [checkersReport]
```

The static analysis passed now, and our code looks like this

```
#include <iostream>
using namespace std;

int* createArray(int size) {
    int* arr = new int[size];
    for (int i = 0; i <= size; i++) { // Off-by-one error
        arr[i] = i * 2;
    }
    return arr;
}

void processData(int* data) {
    cout << "Processing: " << *data << endl;
    delete[] data;
}

int main() {
    int* numbers = createArray(5);
    processData(numbers);
    int x = 2;
    int* ptr = &x;
    *ptr = 10; // Null pointer dereference

    int* leak = new int[100]; // Memory leak
    leak[0] = 42;
    delete [] leak;
    leak = nullptr;

    cout << "Value: " << numbers[0] << endl; // Use after free

    return 0;
}
```

```
mhd@mhd:~/DSA4E/Labs/lab5$ cppcheck --enable=all --suppress=missingIncludeSystem code1.cpp
Checking code1.cpp ...
nofile:0:0: information: Active checkers: 161/592 (use --checkers-report=<filename> to see details) [checkersReport]
mhd@mhd:~/DSA4E/Labs/lab5$
```

No error found by cppcheck, but 2 problems persist, we cannot find them until we run the program

Dynamic Code Analysis

What is Dynamic Analysis?

- Analyzes code WHILE it's running
- Monitors actual program execution and memory usage
- Detects runtime errors that static analysis can't find
- Shows real behavior with actual data

Benefits:

- Catches memory leaks in complex scenarios
- Detects buffer overflows during execution
- Identifies uninitialized memory reads
- Profiles actual memory usage patterns

Dynamic Analysis Tool - Valgrind

There are other tools for this

What is Valgrind?

- Suite of tools for debugging and profiling
- Memcheck: detects memory leaks and errors
- Tracks every memory allocation and access

Installation on Linux:

Ubuntu/Debian

```
sudo apt-get update
```

```
sudo apt-get install valgrind
```

Fedora/RHEL

```
sudo dnf install valgrind
```

Verify installation

```
valgrind --version
```

How to use Valgrind

```
g++ mycode.cpp -o program -g
```

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./program
```

The -g flag is **IMPORTANT**:

- Includes debugging information
- Valgrind shows exact line numbers
- Makes error messages readable

Options explained:

--leak-check=full: Detailed memory leak information

--show-leak-kinds=all: Shows all types of leaks

--track-origins=yes: Shows where uninitialized values came from

We've caught the 2 remaining errors now !!!

```
mhd@mhd:~/DSA4E/labs/lab5$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./code1
==18343== Memcheck, a memory error detector
==18343== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==18343== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==18343== Command: ./code1
==18343==
==18343== Invalid write of size 4
==18343==    at 0x109263: createArray(int) (code1.cpp:7)
==18343==    by 0x1092FF: main (code1.cpp:18)
==18343== Address 0x4e1e094 is 0 bytes after a block of size 20 alloc'd
==18343==    at 0x48485C3: operator new[](unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==18343==    by 0x109237: createArray(int) (code1.cpp:5)
==18343==    by 0x1092FF: main (code1.cpp:18)
==18343==
Processing: 0
==18343== Invalid read of size 4
==18343==    at 0x10937C: main (code1.cpp:29)
==18343== Address 0x4e1e080 is 0 bytes inside a block of size 20 free'd
==18343==    at 0x484BFA4: operator delete[](void*) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==18343==    by 0x1092D7: processData(int*) (code1.cpp:14)
==18343==    by 0x10930F: main (code1.cpp:19)
==18343== Block was alloc'd at
==18343==    at 0x48485C3: operator new[](unsigned long) (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==18343==    by 0x109237: createArray(int) (code1.cpp:5)
==18343==    by 0x1092FF: main (code1.cpp:18)
==18343==
Value: 0
==18343==
==18343== HEAP SUMMARY:
==18343==    in use at exit: 0 bytes in 0 blocks
==18343==    total heap usage: 4 allocs, 4 frees, 75,172 bytes allocated
==18343==
==18343== All heap blocks were freed -- no leaks are possible
==18343==
==18343== For lists of detected and suppressed errors, rerun with: -s
==18343== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)
mhd@mhd:~/DSA4E/labs/lab5$
```


Save code analysis outputs to log files

cppcheck mycode.cpp 2> cppcheck_log.txt

valgrind ./myprogram 2> valgrind_log.txt

Now imagine a program that keeps track of the value of a variable

```
code2.cpp > ...
1  #include<iostream>
2  using namespace std;
3
4  void wahalaSource(int &n){
5      n = 3;
6  }
7
8  void someFunction(int &n){
9      wahalaSource(n);
10 }
11
12 int main(){
13     int x = 5;
14
15     someFunction(x);
16
17     if(x != 5){
18         cout << "Wahala happened !! x = " << x << endl;
19     }
20
21     return 0;
22 }
```

Some useful commands

1. **`gdb ./program`** **`# Start debugging`**
2. **`break main`** **`# Set breakpoint (b main works too)`**
3. **`run`** **`# Start execution (r works too)`**
4. **`next`** **`# Step over function (n works too)`**
5. **`step`** **`# Step into function (s works too)`**
6. **`finish`** **`# Execute until function returns (goes back to caller)`**
7. **`continue`** **`# Continue to next breakpoint (c works too)`**
8. **`print variable`** **`# Print variable value (p works too)`**
9. **`backtrace`** **`# Show call stack (bt works too)`**
10. **`list`** **`# Show source code (l works too)`**
11. **`info breakpoints`** **`# List all breakpoints (i b works too)`**
12. **`delete 1`** **`# Delete breakpoint #1 (d 1 works too)`**
13. **`watch variable`** **`# Break when variable changes`**
14. **`frame 2`** **`# Switch to different stack frame (f 2)`**
15. **`quit`** **`# Exit GDB (q works too)`**

BONUS - Navigation fixes you asked about:

16. **`finish`** **`# Go back to caller (run until current function returns)`**
17. **`reverse-next`** **`# UNDO a next (requires recording mode)`**
18. **`reverse-step`** **`# UNDO a step (requires recording mode)`**